

CPSC 483: Introduction to Machine Learning

Instructor: Anli Ji

Fall 2026

Cal State Fullerton

Chapter 3

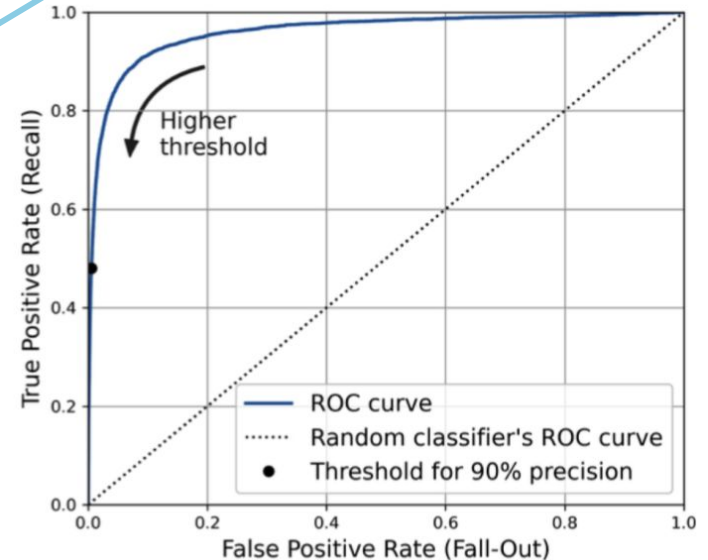
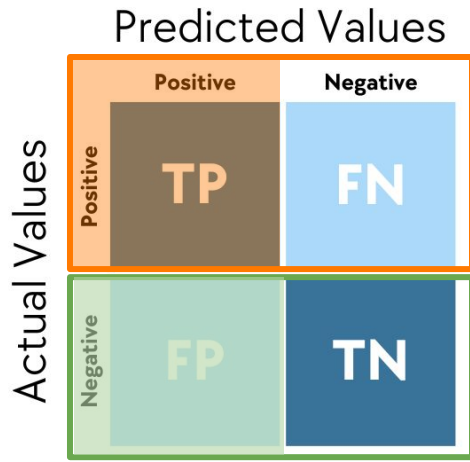
Model Evaluation - Classification

- Binary Classification
- Confusion Matrices
 - Accuracy / Error
 - Confusion Matrix
 - Precision, Recall, F1 Score
 - **ROC / AUC**
- **Multi-class Evaluation**

Receiver Operating Characteristic (ROC) Curve

Similar to the precision/recall curve, but

- instead of plotting precision versus recall
- ROC curve plots **True Positive Rate (TPR/recall)** against **False Positive Rate (FPR)**



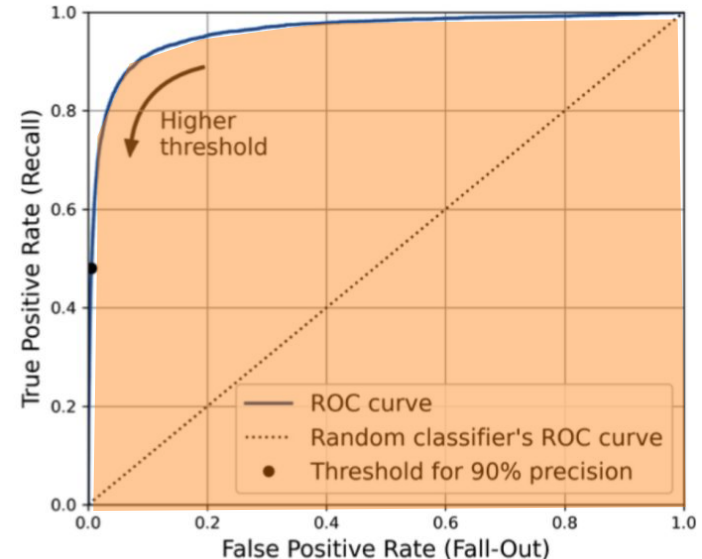
Receiver Operating Characteristic (ROC) Curve

A **good classifier** should be located as close as possible to the upper-left corner, while a random classifier would be along the main diagonal

- Useful for comparing the relative performance among different classifiers

The area under the ROC curve (**AUC**) provides information on which classifier is better on average

- If the model is perfect, its area would be 1
 - random guess = 0.5

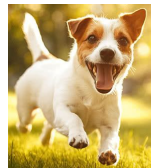
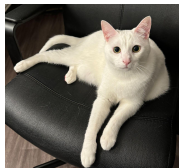


From Binary to Multiclass Classification

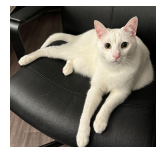
- Binary classifier: distinguish between two classes (Positive and Negative)
- Multiclass classifier: distinguish between more than two classes

Two common strategies:

- **One-versus-All (OvA)** (i.e., One-versus-Rest)
 - Each class competes against all other classes together



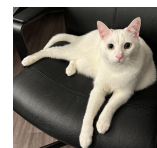
Available Classes



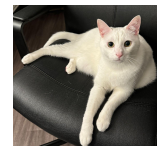
vs.



vs.



vs.

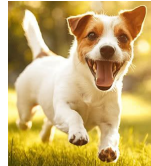


From Binary to Multiclass Classification

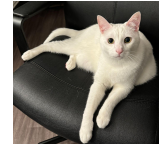
- Binary classifier: distinguish between two classes (Positive and Negative)
- Multiclass classifier: distinguish between more than two classes

Two common strategies:

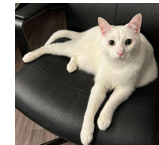
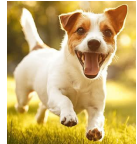
- **One-versus-One (OvO)**
 - Every class competes directly against every other class



Available Classes



vs.



vs.



vs.



Multilabel Classification

Assigns multiple, non-exclusive labels to a single data instance at the same time



[Dog='True', Cat='True', Bird='False', Horse='False']

Chapter 4

Linear Regression

- Simple vs. Multiple
- Regression Performance Measure
- Gradient Descent
 - Batch Gradient Descent
 - Stochastic Gradient Descent
 - Mini-Batch Gradient Descent
- Polynomial Regression

Regression Model

- Relationship between one dependent variable and **explanatory/independent variable(s)**
- Dependent variable is **continuous**
- One equation describes the relationship

Steps to modeling:

- Select the model family
 - Specify the generic pattern of relationship between variables
 - Linear, quadratic, ...
- **Fit** the model to the data
 - Give values to the model parameters
 - Identify the “best” model from the family of models



Linear Regression

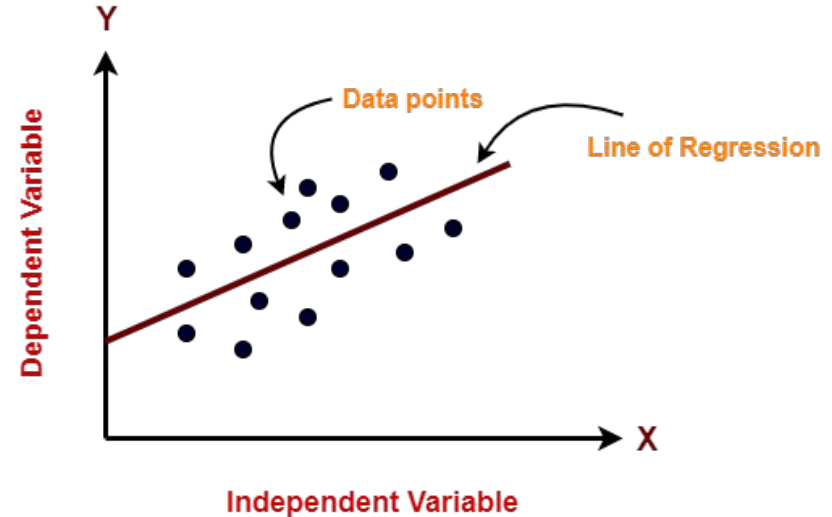
- A statistical and machine learning method used to model the relationship between a dependent target variable and one or more independent explanatory features by fitting a linear equation to observed data

Mathematical speaking...

- **Simple Linear Regression**
 - With one explanatory variable

$$\hat{y} = \theta_0 + \theta_1 x$$

Price (K\$) (Y)	Size (X_1)
375 (y_1)	1024 (x_{11})
425 (y_2)	1329 (x_{21})
...	...
465 (y_N)	1893 (x_{N1})



Linear Regression

- A statistical and machine learning method used to model the relationship between a dependent target variable and one or more independent explanatory features by fitting a linear equation to observed data

Mathematical speaking...

- **Multiple Linear Regression**
 - With multiple explanatory variable

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

```
from sklearn.linear_model import LinearRegression
```

```
lin_reg = LinearRegression()
```

```
lin_reg.fit(X, y)
```

```
lin_reg.intercept_, lin_reg.coef_
```

```
lin_reg.predict(X_new)
```

Price (K\$) (y)	Size (X_1)	Bedroom (X_2)	Bathroom (X_3)	Built year (X_4)
375 (y_1)	1024 (x_{11})	3 (x_{12})	2 (x_{13})	1978 (x_{14})
425 (y_2)	1329 (x_{21})	3 (x_{22})	5 (x_{23})	1992 (x_{24})
...	...	...	...	...
465 (y_N)	1893 (x_{N1})	4 (x_{N2})	4 (x_{N3})	1980 (x_{N4})

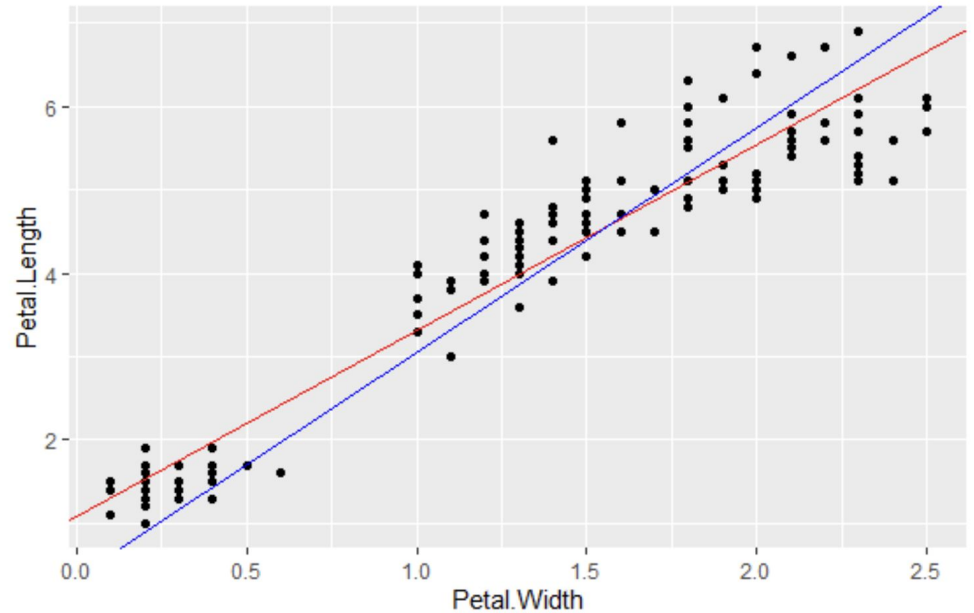
$y_1 = \theta_0 + \theta_1 x_{11} + \theta_2 x_{12} + \theta_3 x_{13} + \theta_4 x_{14}$
$y_2 = \theta_0 + \theta_1 x_{21} + \theta_2 x_{22} + \theta_3 x_{23} + \theta_4 x_{24}$
...
$y_N = \theta_0 + \theta_1 x_{N1} + \theta_2 x_{N2} + \theta_3 x_{N3} + \theta_4 x_{N4}$

in matrix form

$$\begin{bmatrix} 375 \\ 425 \\ \vdots \\ 465 \end{bmatrix} = \begin{bmatrix} 1 & 1024 & 3 & 2 & 1978 \\ 1 & 1329 & 3 & 5 & 1992 \\ \dots & \dots & \dots & \dots & \dots \\ 1 & 1893 & 4 & 4 & 1980 \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_4 \end{bmatrix}$$

Which model/line is better?

Petal. Length	Petal. Width	Output
1.4	0.2	setosa
1.4	0.2	setosa
4.7	1.4	versicolor
6.0	2.5	virginica
...	...	...



Regression Performance Measure

Best Fit

- Find the weights that gives the smallest difference between *Actual Y values* ($y^{(i)}$) & *Predicted Y values* ($\hat{y}^{(i)} = \theta^T x^{(i)}$)
- Differences are called **errors** or **residuals**



Why not adding all residuals together?



Regression Performance Measure

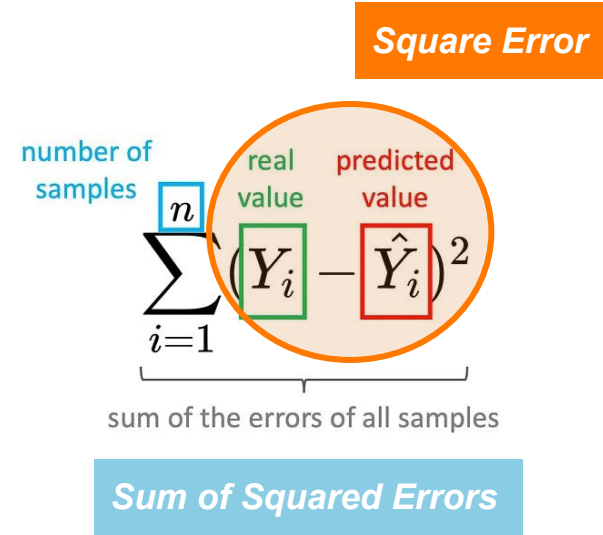
Mean Square Error (MSE)

- A metric used to measure how well a model's predictions match the actual values
- Calculated by taking the average of the squared errors
- Commonly used as a cost function for Linear Regression

Root Mean Square Error (RMSE)

- A variant of MSE that calculates the square root of the average squared difference
- Often preferred over MSE
 - as it provides an interpretable measure of the error in the same units as the original data

$$RMSE = \sqrt{(MSE)}$$



Reference:

<https://levelup.gitconnected.com/understanding-mean-squared-error-a-practical-guide-for-data-scientists-4ec7de03ac0e>

Computational Complexity

- Usually matrix operations for regression
 - i.e., Normal Equation: $\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$

in matrix form

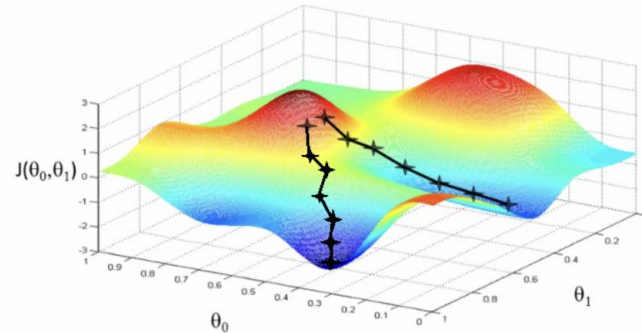
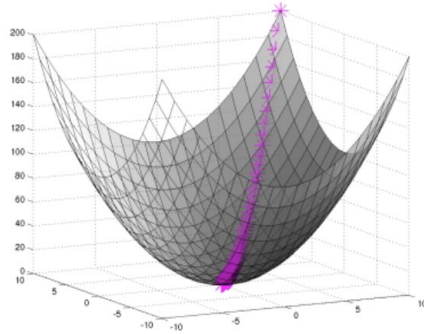
$$\begin{bmatrix} 375 \\ 425 \\ 465 \end{bmatrix} = \begin{bmatrix} 1 & 1024 & 3 & 2 & 1978 \\ 1 & 1329 & 3 & 5 & 1992 \\ \dots & \dots & \dots & \dots & \dots \\ 1 & 1893 & 4 & 4 & 1980 \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \dots \\ \theta_4 \end{bmatrix}$$

- Computationally Expensive
 - Typically $O(n^{2.4})$ to $O(n^3)$, where n is the number of features
 - For example,
 - if double the number of features, simply multiply the computation time by roughly $2^{2.4} = 5.3$ and $2^3 = 8$

Linear Regression as a *Convex Optimization Problem*

Gradient Descent (i.e., steepest gradient descent)

- An iterative optimization approach to find the minimum (or maximum) of a function by taking 1st order derivative



Gradient descent will reach a **global optimum** if a function has only one optimum

- Otherwise, it will reach a **local optimum**

Linear Regression as a *Convex Optimization* Problem

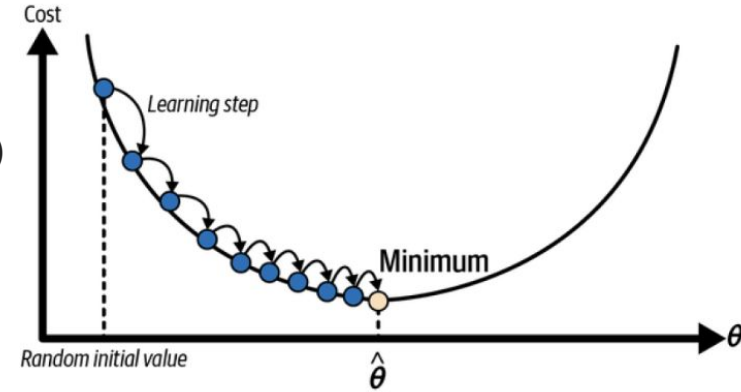
Algorithm wise...

- I. Start with **some initial weights** e.g., $(\theta_0 = 0, \theta_1 = 0)$
- II. **Keep changing** (θ_0, θ_1) **to reduce** $MSE(\theta_0, \theta_1; X, y)$
- III. **Stop when** desirable conditions are satisfied

$$\theta^{(\text{next step})} = \theta - \eta \nabla_{\theta} \text{MSE}(\theta)$$

$$\nabla_{\theta} \text{MSE}(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\theta) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\theta) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y})$$

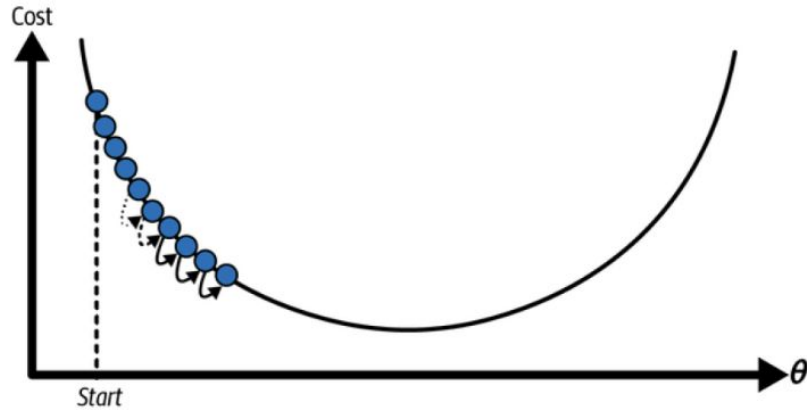
$$\frac{\partial}{\partial \theta_j} \text{MSE}(\theta) = \frac{2}{m} \sum_{i=1}^m \left(\theta^T \mathbf{x}^{(i)} - y^{(i)} \right) x_j^{(i)}$$



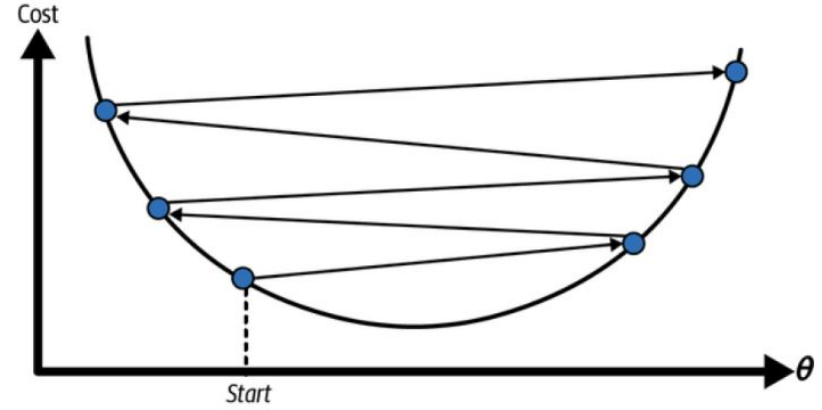
Hyperparameter η is the learning rate to control the speed of convergence

Linear Regression as a *Convex Optimization* Problem

Learning rate...



If too **small**



If too **large**

Variant of Gradient Descent Method

Batch Gradient Descent

- At each step, use ALL the training samples
- Relatively stable and consistent

$$w_j = w_j - \alpha \frac{1}{N} \sum_i (w^T x_i - y_i) x_{ij}$$

Stochastic Gradient Descent (i.e., SGD)

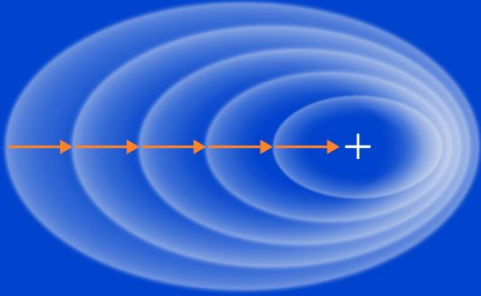
- Use a portion of random training samples at each iteration
- This is effective with big data or for online/adaptive learning

Mini-Batch Gradient Descent

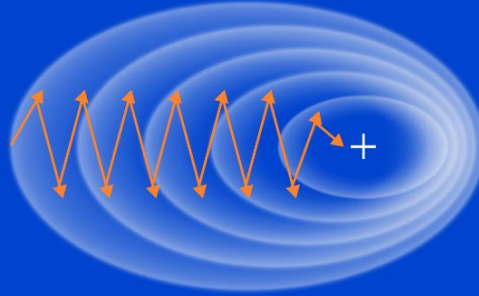
- Combination of batch and SGD
- The training set are split into fixed-size or random batches, then update the weights for each batch
- The training set are then randomly shuffled into new batches and the process is repeated

Variant of Gradient Descent Method

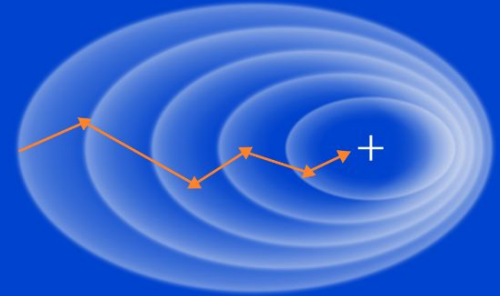
Gradient Descent



Stochastic
Gradient Descent



Mini-Batch
Gradient Descent



Reference:

<https://www.ibm.com/think/topics/stochastic-gradient-descent>